



Somos
F5

Técnicas avanzadas de preprocesamiento o nuevos algoritmos

ABRIMOS RUTAS **INCLUSIVAS**
AL TALENTO DIGITAL



Somos
F5

¿Cuál diríais que es el paso más importante a la hora de entrenar un modelo ML? ¿Por qué?

"Y dentro de la parte de preparar los datos, ¿qué fue lo que más tiempo/quebraderos de cabeza os dio?"

**ABRIMOS RUTAS INCLUSIVAS
AL TALENTO DIGITAL**

GUÍA ESENCIAL DE PREPROCESAMIENTO AVANZADO

// OUTLIERS · ENCODING · FEATURE ENGINEERING

OUTLIERS

deteccion y tratamiento

MÉTODOS DE DETECCIÓN

IQR (rango intercuartílico)

Z-score

Isolation Forest (multivariante)

ESTRATEGIAS DE TRATAMIENTO

Eliminar filas (errores claros)

Capar / Winsorizing (percentil 1-99)

Transformar con logaritmo

ENCODING

categorías -> números

TIPOS PRINCIPALES

One-Hot Encoding - pocas categorías, sin orden

Label Encoding - categorías con orden real

Target Encoding - muchas categorías (riesgo leakage)

FEATURE ENGINEERING

crear nuevas variables

TECNICAS COMUNES

Fechas -> día / mes / fin de semana

Ratios y diferencias entre columnas

Binning - agrupar en categorías

Interacciones entre variables

Preprocesamiento avanzado: OUTLIERS

¿QUÉ SON?

[Más info](#)

Valores que se alejan mucho del resto de los datos

¿POR QUÉ IMPORTAN?

Pueden "engañar" al modelo, especialmente a modelos sensibles a la escala (regresión lineal, KNN¹, redes neuronales).

Los árboles (Random Forest², XGBoost³) son más robustos a ellos.

¿CÓMO DETECTARLOS?

- **IQR (rango intercuartílico):** todo lo que esté fuera de $Q1 - 1.5 \cdot IQR$ y $Q3 + 1.5 \cdot IQR$ se considera outlier.
- **Z-score:** valores con más de ~3 desviaciones estándar de la media.
- **Isolation Forest⁴:** un algoritmo que "aisla" los puntos raros (más avanzado, útil en datasets grandes).

Qué hacer con ellos: eliminarlos, "capar" (winsorizing, limitar al percentil 1-99), o transformar la variable (logaritmo) para reducir su impacto.



Preprocesamiento avanzado: OUTLIERS



Somos
F5



$$Q1 - 1.5IQR \text{ y } Q3 + 1.5IQR$$

Más info

Los conceptos base

- **Q1 (primer cuartil):** el valor por debajo del cual está el 25% de los datos.
- **Q3 (tercer cuartil):** el valor por debajo del cual está el 75% de los datos.
- **IQR = Q3 - Q1:** el rango donde vive el "cuerpo central" de tus datos (el 50% de en medio, sin los extremos).

La regla

Se calculan dos límites:

- **Límite inferior** = $Q1 - 1.5 \times IQR$
- **Límite superior** = $Q3 + 1.5 \times IQR$

Todo dato que caiga **por debajo del límite inferior o por encima del límite superior** se marca como outlier.

Qué

variable (logaritmo) para reducir su impacto.



Z-SCORE

La idea

El Z-score te dice **a cuántas desviaciones estándar está un dato de la media.**

Donde:

- x = el valor del dato
- $\bar{x}$ (x barra) = la media de todos los datos
- σ (sigma) = la desviación estándar

$$Z = \frac{x - \bar{x}}{\sigma}$$

Cómo interpretarlo

- $Z = 0 \rightarrow$ el dato es exactamente igual a la media.
- $Z = 1 \rightarrow$ el dato está 1 desviación estándar por encima de la media.
- $Z = -2 \rightarrow$ el dato está 2 desviaciones estándar por debajo de la media.
- $Z = 3$ o más (en positivo o negativo) $\rightarrow$ dato muy alejado del resto $\rightarrow$ **posible outlier.**

1. Eliminarlo

Quitamos directamente la fila del dataset.

- **Cuándo usarlo:** sospechamos que es un **error de captura de datos** y no un valor real, o cuando hay muchísimos datos y perder unas pocas filas no importa.
- **Riesgo:** si el outlier es un dato real (no un error), pierdes información válida — y si tienes pocos datos, cada fila cuenta.

2. Capar / Winsorizing

En vez de eliminar el dato, se "recorta" a un límite razonable. Por ejemplo, limitar todo al percentil 1 y al percentil 99: si un valor supera el percentil 99, se sustituye por el valor del percentil 99 (no se borra la fila, solo se ajusta ese valor).

- **Cuándo usarlo:** para conservar la fila completa (el resto de columnas de ese dato siguen siendo útiles) pero suavizando el impacto del valor extremo.
- **Ventaja:** no se pierden filas ni información de otras columnas.

3. Transformar la variable (ej: logaritmo)

Cambiar la **escala de toda la columna** (ej: aplicar $\log(x)$), lo que "comprime" los valores muy grandes y reduce su peso relativo.

- **Cuándo usarlo:** típico en variables con distribución muy asimétrica (ej: ingresos, precios de viviendas), donde hay muchos outliers "naturales" en la parte alta, no un par de errores puntuales.
- **Ventaja:** trata el problema de raíz (la distribución sesgada) en vez de tocar dato por dato.
- **Cuidado:** el logaritmo no funciona con valores negativos o cero.

Preprocesamiento avanzado: ENCODING

[Más info](#)

¿EN QUÉ CONSISTE?

Convertir variables categóricas (texto) en números, porque los modelos solo entienden números.

¿POR QUÉ IMPORTAN?

Si no se hace, el modelo directamente no puede procesar esa columna — o peor, si se convierte mal (ej: Label Encoding a categorías sin orden), el modelo puede asumir relaciones falsas (ej: que "azul" > "rojo").

¿CÓMO DETECTARLOS?

- **¿Tiene orden real?** → Label Encoding. ?
- **¿Pocas categorías sin orden?** → One-Hot Encoding⁵ ?
- **¿Muchísimas categorías?** → Target Encoding (con cuidado del data leakage). ?



Preprocesamiento avanzado: ENCODING

¿EN QUÉ CONSISTE?

[Más info](#)

Convertir variables categóricas (texto) en números, porque los modelos solo entienden números.

¿POR QUÉ?

Si no se hace, el modelo puede interpretar relaciones falsas. Ej: "rojo" > "azul".

¿CÓMO?

- **¿Tiene orden real?** → Label Encoding.
- **¿Pocas categorías sin orden?** → One-Hot Encoding⁵.
- **¿Muchísimas categorías?** → Target Encoding (con cuidado del data leakage).

Asigna un número entero a cada categoría (ej. rojo=0, azul=1).

Útil si hay orden; si no, puede confundir al modelo con relaciones falsas.

Preprocesamiento avanzado: ENCODING

¿EN QUÉ CONSISTE?

[Más info](#)

Crea una columna binaria (0/1) por cada categoría.

No asume orden, pero aumenta mucho las columnas si hay muchas categorías.

¿CÓMO DETECTARLOS?

- **¿Tiene orden real?** → Label Encoding. ?
- **¿Pocas categorías sin orden?** → One-Hot Encoding⁵ ?
- **¿Muchísimas categorías?** → Target Encoding (con cuidado del data leakage). ?



Preprocesamiento avanzado: ENCODING

¿EN QUÉ CONSISTE?

[Más info](#)

Reemplaza cada categoría por la media del target para esa categoría.

Potente pero arriesgado: hay que calcularlo solo con datos de train (o con validación cruzada) para evitar que el modelo "vea" el target del test → data leakage.

NOTA: Nunca calcules la media del target usando la misma fila que quieres predecir.

¿CÓMO?

- **¿Tiene orden real?** → Label Encoding. ?
- **¿Pocas categorías sin orden?** → One-Hot Encoding⁵ ?
- **¿Muchísimas categorías?** → Target Encoding (con cuidado del data leakage). ?



¿EN QUÉ CONSISTE?

Crear variables nuevas a partir de las que ya tienes, para darle al modelo información más útil de la que trae el dato en crudo⁶.

¿POR QUÉ IMPORTAN?

Un dato en bruto no siempre expresa bien el patrón real (ej: una fecha no dice mucho, pero "es fin de semana" sí puede explicar un pico de ventas).

Buenas features pueden mejorar el modelo más que cambiar de algoritmo.

¿CÓMO IDENTIFICAR OPORTUNIDADES?

- **¿Hay fechas?** → extraer día/mes/fin de semana
- **¿Hay columnas relacionadas?** → probar ratios o diferencias
- **¿Una variable numérica podría simplificarse?** → binning
- **¿Sospechas que dos variables juntas dicen algo que por separado no?** → crear una interacción.



EL CAMINO HACIA UN MODELO DE ML EXITOSO

// DE DATOS BRUTOS A UN MODELO PREDICTIVO FUNCIONAL

FASE 1 · PREPARACIÓN Y TRANSFORMACIÓN DE DATOS

ANÁLISIS Y LIMPIEZA INICIAL 1

Explorar los datos en profundidad para detectar y tratar outliers que puedan sesgar el modelo.

FEATURE ENGINEERING 2

Crear nuevas variables estratégicas que mejoren el poder predictivo del conjunto de datos.

TRAIN / TEST SPLIT 3

Dividir los datos antes de cualquier ajuste para asegurar una evaluación imparcial del modelo.

FASE 2 · MODELADO Y VALIDACIÓN

ENCODING DENTRO DEL PIPELINE 4

Codificar variables categóricas ajustando el encoder únicamente con los datos de entrenamiento.

ENTRENAMIENTO: BAGGING / BOOSTING 5

Implementar Random Forest, XGBoost u otros algoritmos ensemble según el problema.

EVALUACIÓN DE RESULTADOS 6

Analizar las métricas finales (accuracy, F1-score) para validar la eficacia del modelo.

Bagging (Random Forest)^{7,8} -

[Más info](#)

Varios modelos entrenan **a la vez, cada uno por su cuenta**, sin que se comuniquen entre ellos, y al final se juntan sus respuestas.

¿Cómo se consigue que cada árbol sea distinto?:

- Cada árbol se entrena con una muestra aleatoria de los datos. Esto se llama *bootstrap*.
- En cada árbol se usa solo un subconjunto aleatorio de columnas para decidir cada corte.

Cómo se combina el resultado final:

- Clasificación → votación mayoritaria (gana la especie que más árboles predijeron).
- Regresión → se promedian los números.

Datos originales

```
|— muestra 1 → Árbol 1 → predicción 1  
|— muestra 2 → Árbol 2 → predicción 2  
|— muestra 3 → Árbol 3 → predicción 3  
| ...
```

↓

VOTACIÓN / PROMEDIO

↓

Predicción final

Imagina pedir opinión a 100 personas por separado, y te quedas con lo que dice la mayoría. Aunque algunas personas se equivoquen, el grupo en conjunto suele acertar más que cualquiera de ellas por separado — porque los errores individuales tienden a cancelarse.



Boosting (XGBoost)⁸

[más info](#)

Los modelos entrenan **uno detrás de otro**, en cadena, y cada uno se centra en corregir los errores que cometió el anterior.

Cómo funciona paso a paso:

1. El primer árbol es simple y comete varios errores.
2. El segundo árbol no intenta resolver todo el problema — se centra en los casos donde el primero falló.
3. El tercero se centra en lo que el segundo aún falla.
4. Y así sucesivamente, muchas veces.

Cómo se combina el resultado final:

No es una votación, sino una **suma ponderada** de todos los árboles: cada árbol aporta una pequeña corrección sobre la predicción acumulada.

Árbol 1 → predicción → error 1



Árbol 2 (corrige error 1) → error 2



Árbol 3 (corrige error 2) → error 3



...



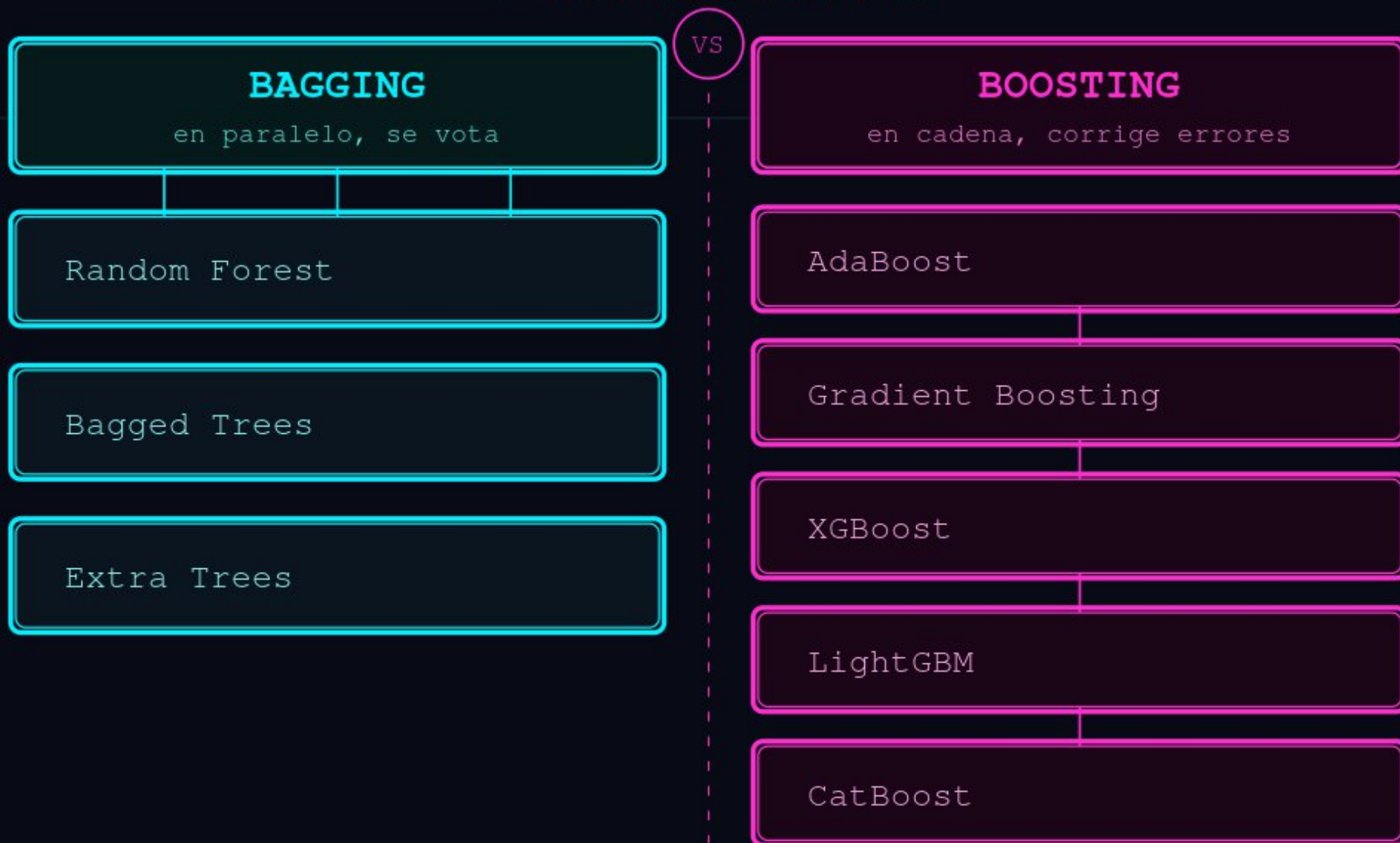
Predicción final (suma de todos)

Es como corregir un examen en varias rondas. La primera corrección detecta los errores más obvios. La segunda revisión se centra solo en lo que quedó mal la primera vez. Cada ronda mejora un poco más el resultado, enfocándose justo donde hacía falta.



ENSEMBLE LEARNING

// FAMILIAS DE ALGORITMOS



RECURSOS RECOMENDADOS



1. Laboratorio 4 - 07/07/2026 - Chat General de Discord
2. Laboratorio 3 - 02/07/2026 - Chat General de Discord
3. [Python: El Detective de Datos, algoritmo xgBoost](#) 
4. [Detección de Datos Anómalos \(outliers\) con Bosques de Aislamiento \(iForests\) y Python](#) 
5. [Aprende a realizar pre-procesamiento de Datos en Python: Variables Dummy \(One Hot Encoding\)](#) 
6. [Feature Engineering en Python | Tutorial Análisis de Datos en Python](#) 
7. [Random Forest \(Bosque Aleatorio\) para Clasificación con Python](#) 
8. [Modelos de conjunto maestro: Bagging vs. Boosting en aprendizaje automático EXPLICADO](#) 

← Volver

🏠 Inicio

ABRIMOS RUTAS **INCLUSIVAS**
AL TALENTO DIGITAL